

Correction — Exercice 7 : DevTools

Solution ● — Réflexes DevTools

Étapes attendues sur la page `inspect.html` :

1. **F12 ouvre les DevTools**. Sous Chrome, on peut aussi clic-droit → *Inspecter*.
2. **Hover sur le DOM** : la zone correspondante s'illumine dans la page. Bleu = content. Vert = padding. Orange = margin.
3. **Computed sur `<h1>`** : `font-size: 32px` (valeur navigateur par défaut pour les `h1`).
4. **Box-model** : pour `<h1>`, on observe un `margin-block-start` et `margin-block-end` non nuls (héritage UA stylesheet).
5. **Outline rouge injecté** : tous les éléments deviennent visibles avec une bordure 1px.

Démonstration de l'astuce `* { outline ... }` : `outline` ne change pas la taille des boîtes (contrairement à `border`). Idéal pour debug visuel.

Solution ● — Inspection guidée

1. **Ajouter padding + bg via inspecteur** : dans le panneau Styles, cliquer dans la zone vide sous `article {...}`, taper `padding: 20px; background: lightyellow`. Le contenu de l'article se décale visuellement.
2. **font-family héritée de `<p>`** : provient de `body` (qui hérite de `html`). Par défaut UA : *Times New Roman* (Chrome/Edge) ou similaire serif. Vérifier dans Computed → expandé `font-family` → la valeur héritée et son origine.
3. **color du `<a>` du `<nav>`** : modification temporaire (live DOM). Au reload, le fichier source est rechargé, le changement disparaît. Pour persister : copier la règle, l'ajouter au CSS source.
4. **Lighthouse** : selon la page, le score Accessibilité de cette page minimale tourne autour de 90-100 (les `<a>` ont du texte, le `<h1>` est présent, lang est défini). Les pertes typiques : pas de `<title>` descriptif, pas de `meta description`.
5. **Network** : `inspect.html` arrive avec :
 - Méthode : `GET`
 - Statut : `200` (ou `304` si cache)
 - Taille : ~500 octets
 - Type : `document`

Solution ● — Bookmarklet de debug

```
javascript:(function(){
  var id = '__debug-outline-style';
  var existing = document.getElementById(id);
  if (existing) {
    existing.remove();
    return;
  }
  var style = document.createElement('style');
  style.id = id;
  style.textContent = '* { outline: 1px solid red !important; outline-offset: -1px !important; }';
  document.head.appendChild(style);
  var divCount = document.querySelectorAll('div').length;
  var totalCount = document.querySelectorAll('*').length;
  alert('Debug ON\n' + divCount + ' <div> détectés\n' + totalCount + ' éléments au total');
})();
```

Installation :

1. Sélectionner et copier toute la ligne ci-dessus.
2. Clic droit sur la barre de favoris → *Ajouter un favori*.
3. Nom : `Debug outline` . URL : coller le code (commençant par `javascript:`).
4. Clic sur le favori pour activer, re-clic pour désactiver.

Points clés expliqués :

- **Toggle via** `getElementById` : le `<style>` est marqué d'un id unique, le re-clic le retrouve et le retire.
- **!important** : nécessaire car certaines pages ont des `outline` déjà définis (ex: Bootstrap sur `:focus`).
- `outline-offset: -1px` : pousse l'outline 1px **vers l'intérieur** pour qu'il ne dépasse pas les bordures voisines.
- `document.querySelectorAll('*').length` : compte total des éléments DOM — utile pour repérer les pages chargées en DOM.

Bonus : variante affichant les microdonnées (schema.org / ARIA) :

```
javascript:(function(){  
  var elements = document.querySelectorAll('[itemscope], [role], [aria-label]');  
  elements.forEach(function(el){  
    el.style.outline = '2px solid green';  
    el.title = (el.getAttribute('role') || '') + ' ' + (el.getAttribute('aria-label') || '');  
  });  
})();
```